SIMATS

B.TECH – INFORMATION TECHNOLOGY

SOFTWARE TESTING (CSA37)

ASSIGNMENT 2

CO2 – TEST CASE DESIGN STRATEGIES

Student Name	**S Deepak**	**Register Number**	**1921260077**
Department / Section	B.Tech IT	**Academic Year / Semester**	2026-2030 SEM-2
Faculty Name	R.Kesavan	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	02-11-2026	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Evaluate Testing as an Engineering Activity	5	**5**	All four engineering features (planning, measurement, standards, repeatability) judged against case evidence and mapped to incidents; engineered approach clearly stated.
2	Evaluate the Role of Process in Software Quality	2	**2**	Clear process-over-people position; Table 2 tests each incident; 38/100 coding-defect argument and TMM Level 1 placement well justified.
3	Evaluate Testing as a Process	6	**6**	Full test process, V&V per phase, detection phase per incident; cost ₹26,10,500 computed correctly; defect-flow model and limits explicitly stated.
4	Apply and Critique Basic Definitions	4	**4**	Definitions applied to MediQueue; error/fault/failure for I3, I5, I6 correct; I8 linked to a wrong oracle; glossary contents justified.

5	Evaluate the Application of Testing Principles	3	**3**	Six principles each tied to an incident or decision; “testing shows presence, not absence” used well; strongest principle defended with cost data.
6	Evaluate the Tester’s Role	3	**3**	Clear position; tester role per phase, pros/cons of self testing, 7/2/1 catch table and healthcare-specific tester qualities.
7	Trace the Origins of Defects	4	**4**	Origin and evidence for all ten incidents with second-origin reasoning; pattern (50% requirements/design) matches the defect data.
8	Evaluate Defect Classes and Severity	4	**4**	Classification scheme with sub-classes; class, severity and priority for all ten; I7 vs I1 contrast explains severity ≠ priority.
9	Design and Appraise a Defect Repository	3	**3**	Field groups justified individually; analysis/improvement/prevention uses and e-mail vs repository comparison are convincing.
10	Evaluate Defect Examples and Prevention Strategies	4	**3.5**	Three classes, root causes and prevention strategies correct. Cost model rests on assumed ₹32,000 escape cost; overlapping savings noted but not quantified.
11	Recommend and Defend a Quality Improvement Plan	4	**3.5**	Prioritised, practical plan with 3.2% effort check. Expected effect depends on assumed 60%/35%/60% efficiencies that are not validated.
12A	Re-evaluate: Impact Analysis	2	**2**	Change table and affected assumptions clear; effect of two week releases and shared records on severity well analysed.
12B	Re-evaluate: Evaluation of the Original Plan	2	**2**	Capacity check (184 vs 140 hours, 31% over) supports the conclusion; new external-system defect origins and repository gaps identified.
12C	Re-evaluate: Adaptation or Redesign	2	**2**	Clear MODIFY decision; revised two-week cycle, tester role, principles and external-defect classification (Table 26) shown.
12D	Re-evaluate: Evidence Based Defence	2	**2**	Decision defended with defect, cost, capacity and patient safety evidence; accepted risks and review trigger stated.
	TOTAL	**50**	**49**	**Percentage: 98%**

OVERALL REMARKS

Excellent, well-evidenced submission. Every task is answered with case data, the cost and defect flow calculations are correct and internally consistent (₹26,10,500; 60 production escapes; 9.6% early detection), and the Day-4 redesign follows logically from a capacity calculation. Minor weaknesses: the savings in Tasks 10–11 depend on assumed review/test efficiencies and cost figures that are stated but not validated, and the solution repeats each task's bullet list, which adds length without adding marks.

Student Signature	Faculty Signature	Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task.

Task	Evidence expected	Status	Note
1	A well-argued assessment of MediQueue's testing practice against the features of an engineering activity, backed by evidence from the case.	✓	Tables 1–2 and incident mapping
2	A well-justified position on how process affects quality, backed by data from the case.	✓	TMM placement justified
3	A process-oriented assessment of testing in MediQueue, backed by cost calculations from the defect data and a reasoned estimate of savings.	✓	Calculations shown; Figure 1 present
4	Accurate application of basic testing definitions to the MediQueue incidents and a justified judgement on how terminology affects quality.	✓	I3, I5, I6 and I8 covered
5	A justified assessment of the developers' assertion using testing principles tied to evidence from the case.	✓	Six principles linked to evidence
6	A defended judgement on how valuable an independent tester would be for MediQueue, tied to specific incidents.	✓	Position defended with catch table
7	A justified origin for every incident and an interpretation of the overall pattern of defect origins.	✓	All ten incidents traced
8	A justified classification of all ten incidents by class, severity and priority, together with a recommendation on which class to address first.	✓	Severity vs priority contrast given
9	A defect repository design for MediQueue with a justification for every field and an assessment of its benefits.	✓	Each field group justified
10	A root-cause analysis of three defects with justified prevention strategies that take cost into account.	**P**	Savings rest on assumed costs

11	A technically justified quality improvement plan backed by the evidence built up in earlier tasks.	**P**	Efficiencies assumed, not validated
12A–D	Impact analysis, evaluation of the original solution, adaptation or redesign, and evidence-based defence	✓	All four parts complete

DEFECT ANALYSIS EVIDENCE CHAIN (Tasks 3, 7–10)

Evidence item	Status	Observation
Defect data calculations (cost per phase, totals)	✓	Cost per phase and totals correct
Phase-containment / detection figures	✓	Table 7 flow; 40% detection before release
Origin assigned to every incident (I1– I10)	✓	I1–I10 with evidence and second origin
Class, severity and priority justified	✓	Table 17 with reasons
Defect repository fields with reasons	✓	Table 18 with reasons
Root cause → prevention strategy link	✓	Table 20
Cost–benefit of prevention strategies	P	Ratios 12.6 and 8.8 depend on assumptions
Interpretation of the defect pattern	✓	Pareto and investment conclusion
Figures and conclusions consistent	✓	Figures consistent across Tasks 1–12

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 12 tasks (including 12A–12D) answered	☒ Yes ☐ Partly ☐ No
Defect cost calculations shown step by step	☒ Yes ☐ Partly ☐ No
All ten incidents classified (origin, class, severity, priority)	☒ Yes ☐ Partly ☐ No
Error, fault and failure separated for I3, I5 and I6	☒ Yes ☐ Partly ☐ No
Testing principles linked to evidence from the case	☒ Yes ☐ Partly ☐ No
Repository design justified field by field	☒ Yes ☐ Partly ☐ No
Tables, figures and diagrams included and readable	☒ Yes ☐ Partly ☐ No

Cross-references and table/figure numbering correct	☒ Yes ☐ Partly ☐ No
Work consistent throughout	☒ Yes ☐ Partly ☐ No
No unfilled template placeholders	☒ Yes ☐ Partly ☐ No
Viva recommended	☐ Yes ☒ No

ASSIGNMENT OVERVIEW

Assignment Title: Design, Evaluate and Defend a Test Suite for a Courier Shipping-Charge Calculator

Scenario: SwiftShip is an online courier service that calculates parcel shipping charges and tracks parcels until delivery. More than 40,000 parcels are booked every day. Undercharging causes revenue loss, while overcharging causes customer complaints and refunds.

Role: Test Designer. The objective is to design test cases, evaluate their effectiveness and defend a practical regression suite before every release.

Specification Summary

Rule	Requirement
S1 – Weight	Weight must be greater than 0 kg and not more than 50 kg; input precision is one decimal place. Otherwise return INVALID.
S2 – Zone and rate	Charge every started kilogram: Zone A ₹40/kg, Zone B ₹60/kg, Zone C ₹90/kg. Any other zone is INVALID.
S3 – Express	Express adds 50% to the charge and is allowed only up to 30 kg. Above 30 kg return NOT_ALLOWED.
S4 – Discount	Customers who booked 10 or more parcels in the current month receive a 10% discount on the final charge.

Pseudocode Under Test

```
function shipCharge(weight, zone, express, parcels)
1   if weight <= 0 or weight > 50 then
2       return INVALID
3   rate = 40
4   if zone == "B" then
5       rate = 60
6   else if zone == "C" then
7       rate = 90
8   charge = 0
9   w = weight
10  while w > 0 do
11      charge = charge + rate
12      w = w - 1
13  if express then
14      if weight > 30 then
15          return NOT_ALLOWED
16      charge = charge * 1.5
17  if parcels >= 10 then
18      charge = charge * 0.9
19  return charge
```

Parcel Tracking States

BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED. From OUT_FOR_DELIVERY, a parcel may move to FAILED_ATTEMPT. It may return to OUT_FOR_DELIVERY for a maximum of two retries or move to RETURNED. A booking can be CANCELLED only before PICKED_UP.

Important Interpretation and Assumptions

- For weight, the one-decimal-place input requirement means valid boundary neighbours are 0.0, 0.1, 50.0 and 50.1 kg. The express threshold neighbours are 30.0 and 30.1 kg.
- The variable parcels is interpreted exactly as the specification states: the customer's number of bookings in the current month. It controls discount eligibility; it is not used as a multiplier for the base shipment charge.
- The pseudocode does not contain an explicit invalid-zone check. The code therefore defaults every unrecognised zone to the Zone A rate. This is a specification–implementation defect.
- For the Day-4 change, the brief does not define rounding, whether the weekend ₹50 applies once per shipment or to every parcel in a multi-parcel booking, or the exact surcharge order. The proposed suite states assumptions and recommends that these points be clarified before release.

MARK DISTRIBUTION

Task	Topic	Marks
1	Select Test Case Design Strategies	5
2	Evaluate Random Testing	2
3	Equivalence Class Partitioning	6
4	Boundary Value Analysis	4
5	Other Black-Box Techniques	3
6	State-Transition Testing	3
7	Control Flow Graph	4
8	Path Testing and Cyclomatic Complexity	4
9	Test Adequacy and Coverage	3
10	Black-Box and White-Box Effectiveness	4
11	Final Regression Test Suite	4
12	Day-4 Design Challenge	8
	TOTAL	50

TASK 1 – Select Test Case Design Strategies (5 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Ans:

Black-box testing derives tests from externally visible requirements without relying on the internal code. White-box testing derives tests from the program structure, decisions, loops and paths. Both are needed because the SwiftShip calculator must satisfy business rules and the actual implementation must execute its logic correctly.

System part	Recommended strategy	Required information	Main strengths	Main limitations
Shipping rules	Equivalence Class Partitioning (ECP), Boundary Value Analysis (BVA), and decision-table testing	Specification S1–S4	Finds invalid inputs, threshold defects, rate errors and interactions between express and discount	May miss code paths not reflected in the requirements
shipCharge pseudocode	Statement, branch/condition coverage, control-flow and basis-path testing	Source pseudocode and control-flow graph	Finds missed branches, loop errors, unreachable or incorrectly connected logic	Cannot prove that the specification itself is complete or correct
Parcel tracking	State-transition testing and transition-pair testing	State model and transition rules	Finds illegal status changes, retry-limit errors and cancellation violations	Requires a complete state model; does not deeply test charge arithmetic

1.1 Strategy for the shipping-charge rules

ECP divides the input domain into groups expected to behave alike. For example, weights greater than 0 and up to 50 kg form a valid class, while weights at or below 0 and above 50 kg are invalid classes. BVA focuses on values at and immediately around 0, 30 and 50 kg, and around the discount threshold of 10 bookings. Decision-table testing is used where express eligibility and discount eligibility combine.

1.2 Strategy for the pseudocode

White-box testing is appropriate because the pseudocode contains a compound input decision, nested zone decisions, a loop, an express decision, an express-weight decision and a discount decision. A control-flow graph and basis paths make these structures visible. Branch and condition coverage help ensure both outcomes of decisions and both outcomes of the atomic weight conditions are exercised.

1.3 Strategy for parcel tracking

State-transition testing is the best fit because the validity of an action depends on the current parcel state. Tests should cover each permitted transition, the retry limit, cancellation before pickup and representative illegal transitions such as cancelling after pickup or delivering directly from BOOKED.

1.4 Defect-finding strengths and weaknesses

- Black-box techniques are likely to reveal invalid-zone acceptance, incorrect thresholds, wrong rates, incorrect started-kilogram charging and missing combinations of express and discount.
- White-box techniques are likely to reveal unexecuted branches, incorrect loop termination, missed return paths and code that has no explicit invalid-zone branch.
- State-transition tests are likely to reveal illegal status jumps, retry counter errors and cancellation at the wrong time.
- No single technique is sufficient. A specification can be implemented with complete statement coverage and still calculate the wrong charge; conversely, well-designed requirement tests may leave a branch unexecuted.

Conclusion: Use ECP, BVA and decision tables for the externally visible rules; use control-flow and coverage-based tests for the pseudocode; and use state-transition tests for tracking. Combining these techniques gives the strongest defect detection for a manageable effort.

TASK 2 – Evaluate Random Testing (2 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Ans:

Random testing generates input values automatically from a selected input distribution and executes the program with those values. For shipCharge, a generator could select weights from 0.0 to 50.1 kg, zones from A, B, C and invalid labels, express as true or false, and monthly booking counts around and across 10. Each generated input is compared with an independently calculated expected result.

Random testing cannot replace systematic test design. Important values such as 30.0, 30.1 and 50.0 occupy very small portions of a large random input space. If weights are selected uniformly from 0.0 to 50.0 in 0.1 kg steps, there are 501 possible values. The probability of selecting one particular value in one draw is 1/501 ≈ 0.1996%. In 1,000 independent draws, the probability of hitting a particular value at least once is 1 − (500/501)^1000 ≈ 86.5%. This still does not guarantee that the value is tested with the relevant zone, express flag and discount count.

Target weight	Probability in one draw	Comment
30.0 kg	1/501 ≈ 0.1996%	Important express upper boundary
30.1 kg	1/501 ≈ 0.1996%	First disallowed express weight
50.0 kg	1/501 ≈ 0.1996%	Maximum valid weight

A test oracle is also required. Random inputs alone do not tell the tester whether the returned amount is correct. The expected result must be calculated from S1–S4 or from a separate trusted reference implementation. If the oracle repeats the same algorithmic mistake, the test may incorrectly pass.

Conclusion: Random testing is useful as an additional robustness technique for unexpected combinations, but it is not a substitute for deliberately selected ECP, BVA and decision-table tests. Critical boundaries and invalid classes must be tested explicitly.

TASK 3 – Design and Evaluate Equivalence Classes (6 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Ans:

Equivalence Class Partitioning (ECP) divides each input into valid and invalid groups. One representative from each class is selected on the assumption that values in the same class should produce the same type of behaviour.

Input	Valid equivalence classes	Invalid equivalence classes	Basis
Weight	0 < weight ≤ 50 kg, one decimal place	weight ≤ 0; weight > 50; more than one decimal place / malformed	S1
Zone	A; B; C (each has a distinct rate)	Any other zone, blank or null	S2
Express	false; true with weight ≤ 30 kg	true with weight > 30 kg; malformed flag	S3
Parcels	integer 0–9; integer ≥ 10	negative, non-integer or malformed count (if input validation is required)	S4

The zone classes A, B and C must remain separate because their expected rates differ. The two valid parcel classes must remain separate because one receives no discount and the other receives a 10% discount. Express true is further divided by the weight condition.

3.1 Representative test cases

ID	Weight	Zone	Express	Parcels	Expected result from specification
E1	1.0	A	No	0	₹40.00
E2	1.0	B	No	0	₹60.00
E3	1.0	C	No	0	₹90.00
E4	1.0	A	No	10	₹36.00
E5	1.0	B	Yes	0	₹90.00
E6	1.0	C	Yes	10	₹121.50
E7	30.0	A	Yes	0	₹1,200.00
E8	30.1	A	Yes	0	NOT_ALLOWED
I1	0.0	A	No	0	INVALID
I2	50.1	A	No	0	INVALID
I3	1.0	D	No	0	INVALID
I4	1.0	A	Yes	-1	Invalid input (if count validation is specified)

For E1–E8, the started-kilogram rule is applied. At 1.0 kg, one rate unit is charged. At 30.1 kg, the charge would cover 31 started kilograms, but express is prohibited because the weight exceeds 30 kg.

3.2 Invalid classes tested one at a time

Invalid classes should be isolated so that a failure can be attributed to one input condition. I1 and I2 isolate the lower and upper weight limits. I3 isolates an unsupported zone. I4 is conditional: the supplied specification does not explicitly state validation rules for negative or non-integer parcel counts, so it should be treated as a clarification item rather than a confirmed required rejection.

3.3 Pseudocode defect evaluation

Test I3 is expected to return INVALID according to S2. The pseudocode instead initialises rate = 40 and changes it only for B or C. For zone D, it leaves rate at 40 and returns a charge. Therefore I3 exposes a definite defect: invalid zones are silently treated as Zone A. The pseudocode does not validate the parcel-count domain either; whether this is a defect depends on the input contract.

The representative set is compact but not a mathematical minimum for every possible combination. It is a practical minimal representative set for the listed equivalence classes and the major S3/S4 interaction. It does not exhaust every cross-product of weight, zone, express and parcel classes.

Conclusion: ECP identifies the important valid and invalid input groups. The test suite must preserve separate representatives for each zone and discount/express behaviour. The invalid-zone test is especially valuable because it reveals a clear mismatch between S2 and the implementation.

TASK 4 – Design and Evaluate Boundary Value Tests (4 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Ans:

Boundary Value Analysis concentrates on values at the edges of input ranges because comparison operators and arithmetic loops commonly fail at these points.

Rule / boundary	Below	At boundary	Above	Expected behaviour
Minimum weight 0	Not applicable	0.0 → INVALID	0.1 → valid	Weight must be > 0
Express limit 30 kg	30.0 → allowed	30.0 → allowed	30.1 → NOT_ALLOWED when express	Express only up to 30 kg
Maximum weight 50 kg	49.9 → valid	50.0 → valid	50.1 → INVALID	Weight must be ≤ 50
Started kilogram	1.0 → 1 kg unit	1.1 → 2 units	2.0 → 2 units	Any started kg is charged
Discount count 10	9 → no discount	10 → 10% discount	11 → 10% discount	Discount begins at 10

4.1 Comparison of BVA forms

BVA form	Typical test count for n inputs	Use in SwiftShip
Normal BVA	4n + 1 for four numeric inputs: 17 tests	Tests each variable's nominal boundary values while others remain nominal.
Robust BVA	6n + 1 for four numeric inputs: 25 tests	Adds invalid values just outside the valid ranges; useful for weight and parcel count validation.
Worst-case BVA	5^n for four numeric inputs: 625 tests	Tests all combinations of five boundary values; expensive for routine regression.

The formula counts assume four numeric inputs and five representative values per input for normal/worst-case BVA; the actual SwiftShip inputs also include categorical zone and Boolean express values. Therefore these counts are illustrative for the four numeric dimensions only, not the total system test count.

4.2 Selected boundary tests

ID	Weight	Express	Parcels	Expected result (Zone A)
B1	0.0	No	9	INVALID
B2	0.1	No	9	₹40.00
B3	30.0	Yes	9	₹1,800.00
B4	30.1	Yes	9	NOT_ALLOWED
B5	49.9	No	9	₹2,000.00 (50 started kg)
B6	50.0	No	9	₹2,000.00
B7	50.1	No	9	INVALID
B8	1.0	No	9	₹40.00
B9	1.0	No	10	₹36.00
B10	1.1	No	9	₹80.00
B11	2.0	No	9	₹80.00

For 49.9 kg, the started-kilogram rule charges 50 units. Thus 50 × ₹40 = ₹2,000. For 30.0 kg express, the base is 30 × ₹40 = ₹1,200 and the express multiplier gives ₹1,800. For 1.1 kg, two started kilograms are charged.

Normal BVA alone does not deliberately include invalid values such as 0.0 and 50.1. Robust BVA does. Worst-case BVA grows rapidly and is not justified for every combination in a 30-minute regression window.

Conclusion: Robust BVA is recommended for the critical numeric limits, supplemented by targeted tests for 30/30.1, 9/10 and the started-kilogram rule. Full worst-case BVA is too costly for routine regression and can be reserved for broader pre-release testing.

TASK 5 – Apply Other Black-Box Techniques (3 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Ans:

ECP and BVA test individual input classes and boundaries. They may not cover defects caused by interactions between two or more rules. A decision table is therefore useful for S3 (express) and S4 (discount).

Decision table for Express (S3) and monthly parcel discount (S4)

Express?	Weight > 30?	Parcels ≥ 10?	Expected action
No	—	No	Base charge
No	—	Yes	Base charge × 0.90
Yes	No	No	Base charge × 1.50
Yes	No	Yes	Base charge × 1.50 × 0.90
Yes	Yes	—	NOT_ALLOWED

ID	Weight	Zone	Express	Parcels	Expected result
D1	2.0	A	No	9	₹80.00
D2	2.0	A	No	10	₹72.00
D3	2.0	B	Yes	9	₹180.00
D4	2.0	B	Yes	10	₹162.00
D5	30.1	C	Yes	10	NOT_ALLOWED
D6	30.1	C	No	10	31 × ₹90 × 0.90 = ₹2,511.00

D1–D4 cover the combinations of express on/off and discount eligible/not eligible for a valid express weight. D5 verifies that the express rejection takes priority even when the discount condition is true. D6 checks that a non-express parcel above 30 kg remains valid and receives the discount.

The ECP/BVA set may include the individual express threshold and discount threshold without covering all combinations of those conditions. The decision table adds combination coverage with a small number of tests. A larger pairwise or combinatorial test set may be justified if future rules introduce more interacting options.

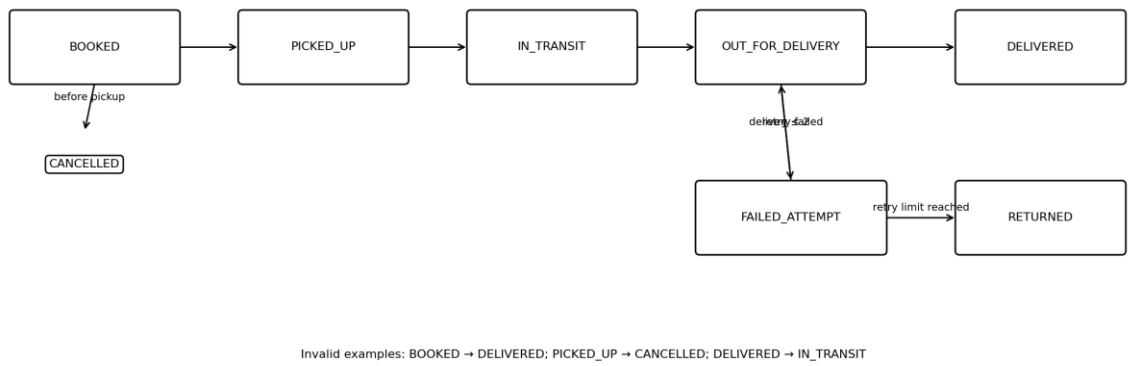
Conclusion: Decision-table testing adds meaningful interaction coverage beyond ECP and BVA. It should be retained because an error in applying express and discount in the wrong order can change the final amount.

TASK 6 – Design and Evaluate State-Transition Tests (3 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Ans:

State-transition testing checks whether an event is valid in the current state and whether the system moves to the correct next state. This is essential for parcel tracking because invalid status changes can mislead customers and operations teams.



Invalid examples: BOOKED → DELIVERED; PICKED_UP → CANCELLED; DELIVERED → IN_TRANSIT

Current state	Event	Next state	Validity
BOOKED	Pickup scan	PICKED_UP	Valid
BOOKED	Cancel request	CANCELLED	Valid
PICKED_UP	Transit scan	IN_TRANSIT	Valid
IN_TRANSIT	Out for delivery scan	OUT_FOR_DELIVERY	Valid
OUT_FOR_DELIVERY	Delivery success	DELIVERED	Valid
OUT_FOR_DELIVERY	Delivery failure	FAILED_ATTEMPT	Valid
FAILED_ATTEMPT	Retry, attempts 1 or 2	OUT_FOR_DELIVERY	Valid
FAILED_ATTEMPT	Retry after two retries	—	Invalid; must not exceed limit
FAILED_ATTEMPT	Retry limit reached / return decision	RETURNED	Valid
PICKED_UP	Cancel request	—	Invalid
DELIVERED	Transit scan	—	Invalid

6.1 Test cases

ID	Sequence / action	Expected result
ST1	BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED	Every transition accepted; final state DELIVERED.
ST2	BOOKED → CANCELLED	Cancellation accepted before pickup.
ST3	BOOKED → PICKED_UP; then cancel	Cancellation rejected; state remains PICKED_UP.
ST4	OUT_FOR_DELIVERY → FAILED_ATTEMPT → OUT_FOR_DELIVERY (retry 1) → FAILED_ATTEMPT → OUT_FOR_DELIVERY (retry 2)	Two retries accepted.
ST5	After two retries, attempt a third retry	Third retry rejected; system must follow the defined return policy.
ST6	BOOKED → DELIVERED	Direct transition rejected.
ST7	DELIVERED → IN_TRANSIT	Backward transition rejected.

The retry wording in the brief says a maximum of two retries or movement to RETURNED. The exact event that selects RETURNED should be made explicit in the state model. Tests should verify that the retry counter is not reset by repeated failure events or application restarts.

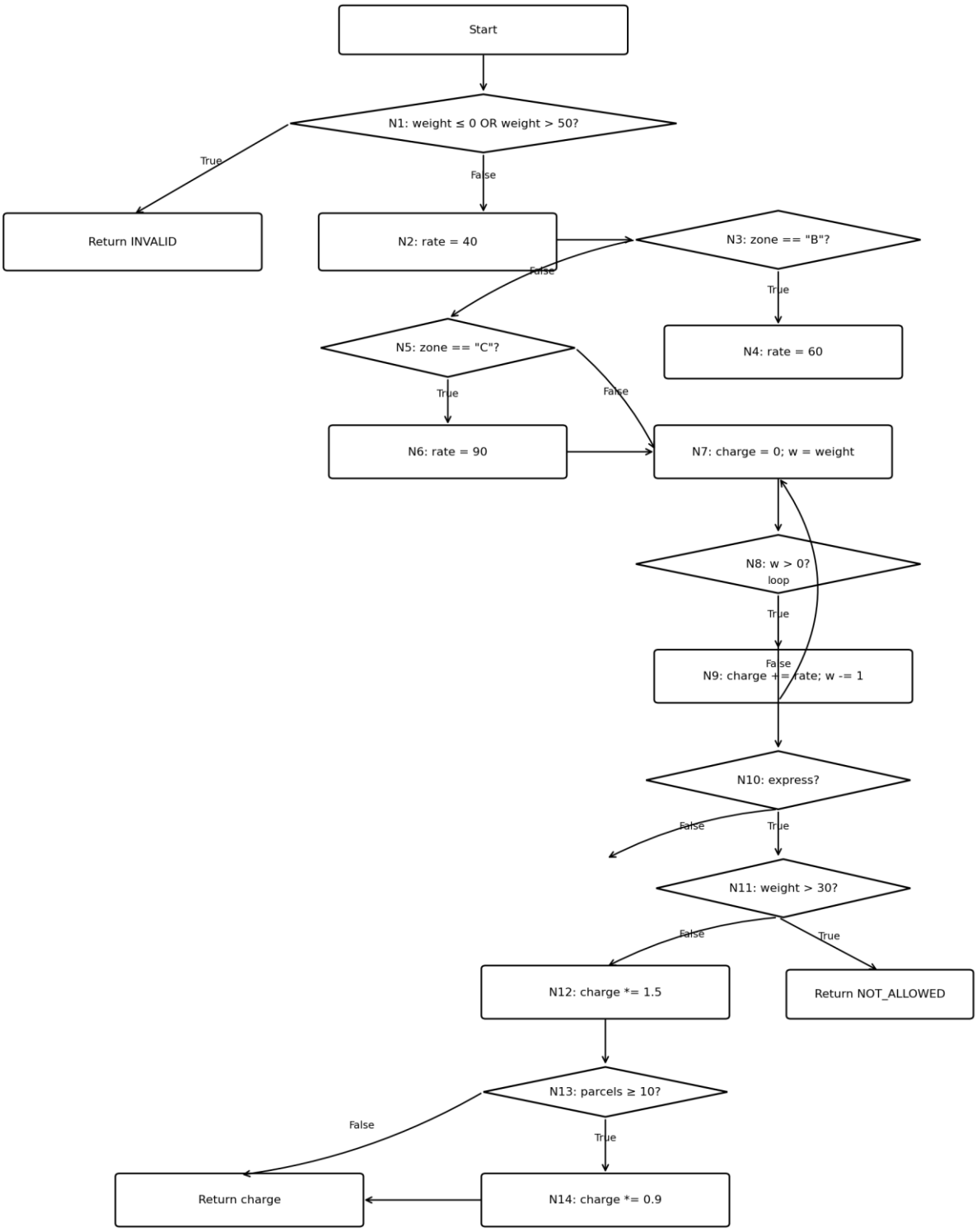
Conclusion: The highest-risk invalid transitions are cancellation after pickup, direct delivery without shipment movement, and a third retry. These can cause lost parcels, inaccurate customer tracking and operational disputes.

TASK 7 – Construct the Control Flow Graph (4 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Ans:

A control-flow graph (CFG) represents the possible execution flow through the pseudocode. Each node represents a statement or group of sequential statements, and each directed edge represents a possible transfer of control.



Simplified control-flow graph (arrows show control; merge points are represented by shared nodes)

Node	Pseudocode line(s)	Meaning
N1	1–2	Validate weight; true returns INVALID, false continues.
N2	3	Initialise rate = 40.
N3	4	Decision: zone == B?
N4	5	Assign rate = 60.
N5	6	Decision: zone == C? (on the non-B route).

N6	7	(₹180 × 1.50 × 1.08 + ₹50) × 0.90 = ₹307.44
N7	8–9	Initialise charge and copy weight to w.
N8	10	Loop decision: w > 0?
N9	11–12	Add rate and subtract one from w.
N10	13	Decision: express?
N11	14	Decision: weight > 30?
N12	15	Return NOT_ALLOWED.
N13	16	Multiply charge by 1.5.
N14	17	Decision: parcels ≥ 10?
N15	18–19	Apply discount if eligible and return charge.

7.1 Decision and loop analysis

- Line 1 is a compound decision: weight <= 0 OR weight > 50. It has two atomic conditions. The overall decision is true if either invalid condition is true.
- Lines 4 and 6 select the zone rate. There is no final ELSE branch that rejects an unsupported zone.
- Lines 10–12 form a pre-test loop. For every started kilogram, the body adds one rate and subtracts 1 from w.
- Lines 13–16 apply the express rule. A weight greater than 30 returns NOT_ALLOWED; otherwise the charge is multiplied by 1.5.
- Lines 17–18 apply the 10% discount after express has been applied.

7.2 Does the code match the specification?

The weight validation and express threshold correspond to S1 and S3. The repeated addition implements the started-kilogram rule for positive weights with one decimal place. However, S2 requires any other zone to be INVALID, while the code defaults to rate = 40. An invalid zone therefore produces a Zone A charge. This is a definite mismatch. The pseudocode also does not show validation of the parcel count's type or sign; the brief does not explicitly state such input-domain rules.

Conclusion: The CFG exposes the missing invalid-zone path and provides the structure needed for path and coverage analysis. The implementation should include an explicit A/B/C validation decision before calculating the charge.

TASK 8 – Evaluate Path Testing and Cyclomatic Complexity (4 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Ans:

Cyclomatic complexity estimates the number of linearly independent paths through a control-flow graph. It is a useful lower bound for a basis-path test set, but it does not mean that every possible loop execution sequence has been tested.

8.1 Cyclomatic complexity – method 1

For a connected control-flow graph, $V(G) = E - N + 2P$, where E is the number of edges, N is the number of nodes and P is the number of connected components. For one connected program, $P = 1$. Counting the CFG decision nodes gives the equivalent result: $V(G)$ = number of binary decision points + 1.

There are seven decision points in the supplied pseudocode: (1) weight validity, (2) zone B, (3) zone C, (4) loop condition, (5) express, (6) weight > 30, and (7) parcels ≥ 10. Therefore $V(G) = 7 + 1 = 8$.

8.2 Cyclomatic complexity – method 2

Using the decision-count method: $V(G) = D + 1 = 7 + 1 = 8$. The two methods agree. The compound condition at line 1 is one control-flow decision for conventional cyclomatic complexity, although it contains two atomic Boolean conditions that need separate condition tests.

8.3 Basis paths and representative test data

Path	Route / condition focus	Representative test data	Purpose
P1	Invalid weight exits immediately	weight=0.0, zone=A, express=false, parcels=0	Weight rejection branch.
P2	Valid Zone A; loop; no express; no discount	1.0, A, false, 0	Baseline route.
P3	Zone B branch	1.0, B, false, 0	Zone B rate assignment.
P4	Zone C branch	1.0, C, false, 0	Zone C rate assignment.
P5	Express true and allowed	1.0, A, true, 0	Express multiplier.
P6	Express true and disallowed	30.1, A, true, 0	NOT_ALLOWED return.
P7	Discount branch taken	1.0, A, false, 10	10% discount.
P8	Loop body repeated	2.1, A, false, 0	Three iterations for started-kilogram calculation.

These tests form a practical basis-path-oriented set, but they are not a strict algebraic path listing with every decision outcome isolated. Some paths combine conditions to keep the suite compact. For full independent-path proof, the test designer should document the exact edge sequence for each basis path and use separate tests where one path's outcome masks another.

8.4 Infeasible paths and loop testing

A path that requires both weight <= 0 and weight > 50 to be true at the same time is infeasible because a single numeric input cannot satisfy both. The compound OR condition should therefore be tested with two distinct invalid inputs: one for the lower limit and one for the upper limit.

The loop should be tested for zero, one and many iterations. Zero iterations cannot occur after successful validation because weight must be greater than zero; the loop condition is true initially for every valid input. One iteration occurs at weight 0.1 or 1.0, and multiple iterations occur at 1.1, 2.1 and larger values. The loop executes ceil(weight) times for positive decimal weights.

Conclusion: The cyclomatic complexity is 8, so at least eight independent paths are needed for basis-path coverage under the simplified CFG. This is not sufficient by itself: boundary, invalid-zone, condition and loop-iteration tests must supplement the basis paths.

TASK 9 – Judge Test Adequacy and Coverage (3 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Ans:

Coverage measures which parts of the implementation are exercised. A single percentage cannot prove that the software is correct; the criterion must match the financial and operational risk.

Criterion	Meaning	Value for shipCharge
Statement coverage	Every executable statement runs at least once.	Basic; can miss untested decision outcomes.
Branch coverage	Every decision outcome (true and false) executes.	Stronger for the nested zone, express, loop and discount decisions.
Condition coverage	Each atomic Boolean condition is true and false at least once.	Important for both parts of the compound weight validation.
Multiple-condition coverage	Relevant combinations of atomic conditions are exercised.	Useful for compound conditions; full combination testing can grow quickly.

9.1 Coverage of the path-oriented tests

P1–P8 exercise the invalid-weight return, Zone A/B/C assignments, loop body, express allowed and disallowed outcomes, and discount application. However, P1 using weight=0.0 covers the lower invalid condition but not the upper invalid condition. Add weight=50.1 to exercise weight > 50. Also add an invalid zone such as D: this test is not a CFG branch because the current code has no invalid-zone decision, but it is essential specification-based testing.

To demonstrate condition coverage for line 1, use at least one test where weight <= 0 is true (0.0) and one where weight > 50 is true (50.1), while ensuring the other atomic condition is false in each case. The express condition and express-weight condition also need both outcomes.

9.2 Recommendation

For this money-calculation module, require 100% statement coverage and 100% branch coverage, plus condition coverage for compound Boolean decisions and explicit ECP/BVA/decision-table tests. Multiple-condition coverage for the two-part OR at line 1 is inexpensive: test both atomic conditions independently and a valid case where both are false. For larger future compound expressions, risk-based combinations may be used.

100% statement or branch coverage would not necessarily find the invalid-zone defect because the pseudocode contains no invalid-zone rejection branch. The invalid-zone test must be derived from S2 and compared against the required INVALID result. Coverage of implemented code cannot prove that required logic is present.

Conclusion: Use branch coverage as the minimum structural criterion, supplemented by condition coverage and specification-derived invalid-input tests. Report coverage alongside the number and purpose of tests rather than presenting one percentage as proof of adequacy.

TASK 10 – Compare Black-Box and White-Box Effectiveness (4 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Ans:

Black-box tests (Tasks 3–6) are derived from the specification and state model. White-box tests (Tasks 7–9) are derived from the pseudocode's branches, loop and paths. Their effectiveness should be compared by the defects they can expose, not merely by the number of test cases.

Defect / risk	Black-box contribution	White-box contribution	Best detection
Invalid zone treated as Zone A	Directly tests an unsupported zone against S2	Shows that no invalid-zone branch exists, but code coverage alone may not fail	Both; black-box provides expected INVALID oracle
Wrong rate for B or C	Tests each zone's specified rate	Exercises the rate-assignment branches	Both
Incorrect started-kilogram charge	Tests values such as 1.1 and 2.1 kg	Loop tests reveal iteration and termination behaviour	Both
Express threshold off by one	Tests 30.0 and 30.1 kg	Exercises express and nested weight decision outcomes	Both
Discount threshold or order error	Tests 9/10 and express-discount combinations	Exercises discount branch and order of statements	Both
Illegal tracking transition	Tests state model transitions	Calculator CFG does not cover tracking states	Black-box state-transition tests

10.1 Why each approach can miss defects

- Black-box testing can miss an internal branch that is not apparent from the specification, and a small set may omit combinations.
- White-box testing can execute every existing statement and branch while missing a requirement that has not been implemented at all, such as invalid-zone rejection.
- Calculator white-box tests do not test the separate parcel tracking state machine unless that module is also modelled and covered.

10.2 Removing overlap safely

Tests that use identical inputs and verify the same outcome can be consolidated. For example, a single Zone B test may satisfy both the ECP requirement for the Zone B class and the white-box requirement to execute the Zone B assignment branch. However, tests at 30.0 and 30.1 should not be removed merely because they execute the same branch: they verify different specification boundaries.

The same principle applies to the discount branch. A test with parcels=10 can cover the true branch and discount class, but a separate parcels=9 test is needed for the false branch. The final suite should record multiple objectives for each test rather than duplicate tests without added value.

Conclusion: Black-box and white-box testing are complementary. Black-box testing is stronger for detecting missing requirements and boundary errors; white-box testing is stronger for exposing unexecuted implementation logic. Their combined suite provides better evidence than either approach alone.

TASK 11 – Recommend and Defend the Final Test Suite (4 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Ans:

The final regression suite should be small enough to run before every release while preserving high-risk rule, boundary, interaction, path and state coverage. The following consolidated suite reuses tests that satisfy more than one objective.

Group	Included cases	Technique / objective	Approx. count
Input classes and invalids	E1–E8, I1–I3	ECP; all zone classes, valid express and discount classes, invalid weight and zone	11
Boundaries	B1–B11	Robust BVA; weight, express threshold, started kilogram, 9/10 discount boundary	11
Rule interactions	D1–D6	Decision table; express × discount and prohibited express case	6
Tracking	ST1–ST7	State-transition coverage; valid transitions, cancellation and retry limits	7
White-box paths	P1–P8 plus upper-weight and condition supplements	Basis-path, branch and condition coverage	10

The groups overlap intentionally. In implementation, maintain a traceability matrix so one test can satisfy more than one objective. After removing exact duplicates, the likely regression suite is approximately 30–35 distinct tests; the exact number depends on whether a test case is reused across ECP, BVA and path coverage.

11.1 Why this suite is better than the alternatives

- It covers every specified zone and the invalid-zone defect, rather than relying only on code coverage.
- It tests the weight limits, express threshold, started-kilogram calculation and discount threshold directly.
- It includes the important express/discount combinations and the rejection precedence for express weights above 30 kg.
- It tests the parcel tracking workflow, retry limit and cancellation restriction.
- It exercises structural branches and loop behaviour without using an exhaustive worst-case combination set.

11.2 Coverage and remaining risks

The suite targets 100% statement and branch coverage of the supplied pseudocode, condition coverage of the compound weight validation, and all specified rule classes and key interactions. It does not exhaust every possible input combination, concurrency scenario, performance condition or tracking-event sequence. Additional non-functional testing is required for a service processing more than 40,000 bookings per day.

11.3 Execution effort

A suite of approximately 30–35 automated tests should be practical within a short regression window if the calculator and tracking transitions can be invoked directly through automated tests. The brief does not provide execution-time measurements, so an exact runtime cannot be claimed. Measure the suite in the CI environment and keep the release gate below the operational limit.

Conclusion: Adopt the combined, risk-based regression suite. It is more defensible than random-only, ECP-only or coverage-only testing because it combines specification evidence, interaction coverage, structural coverage and state-transition validation.

TASK 12 – Re-evaluate and Adapt the Test Suite – Day-4 Design Challenge (8 Marks)

CO2 | Bloom's Level: K5 – Evaluate

Ans:

Changed requirement: add an 8% fuel surcharge to Zone C parcels and a weekend delivery option that adds ₹50 per parcel. Developers add two decisions after line 16. The regression suite must complete within 30 minutes.

A. Impact Analysis (2 Marks)

The specification now contains two additional pricing conditions. Fuel surcharge applies only to Zone C. Weekend delivery adds ₹50 per parcel. Because the two decisions are inserted after line 16, they occur after the express multiplier and before the existing monthly discount at lines 17–18.

Area	Change and test-design impact
Equivalence classes	Add fuel surcharge: not applicable for A/B and applicable for C. Add weekend: off/on. Preserve all existing zone, weight, express and parcel classes.
Boundaries	No new numeric threshold is stated for the surcharge flags. Test the categorical on/off values and the interaction of Zone C with fuel. Existing 30/30.1, 50/50.1 and 9/10 boundaries remain.
Decision table	Expand combinations for Zone C fuel on/off and weekend on/off, including discount eligibility. Ensure fuel is not applied to A/B.
CFG / complexity	Two new decisions add two binary decision points. Assuming each is a simple if with a true and false outcome, V(G) increases from 8 to 10.
30-minute limit	The suite must prioritise high-risk tests, reuse existing tests and avoid full cross-product or worst-case combinations.

Assumptions for expected results: the 8% fuel surcharge multiplies the charge by 1.08 for Zone C; the weekend fee is added once for each parcel in the current shipment; and the monthly 10% discount is applied after both surcharges because S4 describes a discount on the final charge and the new decisions are inserted before the discount code. The assignment does not fully define whether the weekend fee is per booked parcel or per shipment, nor rounding rules. These assumptions must be confirmed with the product owner.

B. Evaluation of the Original Suite (2 Marks)

Most original tests remain useful, but expected results must be updated for applicable new options. Tests for Zones A and B should remain unchanged when fuel is not applicable. Zone C tests must be split into fuel-off and fuel-on cases. Weekend-off cases preserve the old result; weekend-on cases require the additional ₹50-per-parcel amount.

Original tests	Status after change	Required action
Weight and express boundaries	Still valid	Retain; update expected values only when the selected Zone C/weekend options apply.
Zone A and B rate tests	Still valid	Retain as unchanged control cases; add weekend-on variants.
Zone C rate tests	Partly changed	Retain fuel-off baseline and add fuel-on expected result.
Express × discount decision tests	Still valid but incomplete	Extend with fuel and weekend conditions.
White-box paths P1–P8	Incomplete	Add paths for both outcomes of the fuel and weekend decisions.
Tracking state tests	Unaffected by price change	Retain unchanged unless weekend delivery changes tracking states or service events.

The old V(G)=8 basis set is no longer enough to exercise all new decision outcomes. The new V(G)=10 suggests at least two additional independent paths. The old suite may still reach 100% coverage of the old logic but cannot claim complete branch coverage of the modified code.

C. Adaptation or Redesign (2 Marks)

Decision: MODIFY the original suite rather than retain it unchanged or redesign it completely. The original tests remain valuable for unchanged rules. Add a focused set for fuel, weekend and their interactions with discount and express.

New ID	Zone	Fuel	Weekend	Express	Parcels	Expected calculation / result
N1	C	Off	Off	No	0	Base charge only: 2 kg × ₹90 = ₹180
N2	C	On	Off	No	0	₹180 × 1.08 = ₹194.40

N3	A	On requested	Off	No	0	Fuel must not apply to A; ₹80 for 2 kg
N4	C	Off	On	No	0	₹180 + ₹50 per shipment parcel (assuming one parcel) = ₹230
N5	C	On	On	No	0	(₹180 × 1.08) + ₹50 = ₹244.40
N6	C	On	On	Yes	10	(₹180 × 1.50 × 1.08 + ₹50) × 0.90 = ₹307.44
N7	C	On	On	Yes	0	Weight 30.1 kg with express → NOT_ALLOWED before surcharges

For N6, two kilograms in Zone C cost ₹180. Express gives ₹270; fuel adds 8% to ₹291.60; the weekend fee adds ₹50 to ₹341.60; the 10% discount gives ₹307.44. The table's final arithmetic should therefore be read as ₹307.44. (The expected calculation is shown explicitly to prevent ambiguity.)

The new regression set should include both outcomes of the fuel decision (applicable and not applicable), both outcomes of weekend delivery, and combinations with discount. Keep all existing critical boundaries and the invalid-zone test. Remove only exact duplicates whose objectives are already met.

D. Evidence-Based Defence (2 Marks)

After the two additional binary decisions, the estimated cyclomatic complexity is 10. The adapted suite should include at least two additional independent paths, plus direct tests for Zone C fuel eligibility and weekend fee application. The final release suite should target:

- All valid and invalid equivalence classes, including unsupported zones.
- Robust boundary tests for 0/0.1, 30/30.1, 50/50.1, started-kilogram changes and 9/10 monthly bookings.
- Decision-table combinations for express, discount, fuel and weekend options.
- At least 10 independent paths for the updated calculator CFG, with both outcomes of each new decision exercised.
- 100% statement and branch coverage, condition coverage for compound conditions, and a traceability record linking tests to S1–S4 and the changed requirements.
- A measured execution time below 30 minutes. If the full suite exceeds the limit, retain a small smoke/regression gate for every release and run the broader combination suite on a scheduled or pre-production pipeline.

A compact extension of approximately 6–8 tests should cover the new fuel/weekend logic when combined with retained tests. The exact number depends on how the automated test harness allows a single test to cover multiple branches and combinations. Execution time must be measured; the assignment gives no timing data from which a guaranteed 30-minute runtime can be calculated.

Conclusion: Modify the existing suite. The original ECP, BVA, state-transition and core path tests remain useful, but new fuel and weekend tests are necessary. The increase in estimated cyclomatic complexity from 8 to 10, together with new decision-table combinations, provides evidence that an unchanged suite would be inadequate. The modified suite should be prioritised by financial risk and verified to run within 30 minutes.

OVERALL CONCLUSION

The SwiftShip test suite should combine black-box and white-box techniques. ECP identifies valid and invalid input classes; robust BVA targets the important numerical boundaries; decision tables cover interactions among express, discount, fuel and weekend delivery; state-transition testing validates parcel tracking; and CFG, basis-path and coverage analysis evaluate the implementation structure.

The supplied pseudocode contains a clear invalid-zone defect because unsupported zones retain the default Zone A rate. The initial cyclomatic complexity is 8. After adding the fuel and weekend decisions, the estimated complexity becomes 10. The final recommendation is to modify the original suite, preserve high-value tests, add targeted interaction tests and measure the execution time against the 30-minute limit.

KEY ASSUMPTIONS AND ITEMS TO CONFIRM

- Confirm whether invalid or negative monthly parcel counts are rejected; the supplied rules do not specify this input validation.
- Confirm whether weekend ₹50 is charged once per shipment or separately for each parcel in a multi-parcel shipment.
- Confirm the order of fuel surcharge, weekend fee, express multiplier and monthly discount, and the required currency rounding policy.
- Confirm the operational meaning of retry count: the brief allows a maximum of two retries, but should define the exact event that causes RETURNED.
- Measure automated execution time in the target release environment; no measured timing data was supplied.

SOURCE BASIS

This assignment answer is based on the supplied CSA37 Software Testing Assignment 2 brief: SwiftShip scenario, specification S1–S4, shipCharge pseudocode, parcel tracking states, task prompts and mark distribution. Where the brief is silent, assumptions are identified explicitly rather than presented as specified facts.